tmux documentation

tracker_structural_integrity.sh — tracker section/preamble survival guard

Revision: 1 **Last modified:** 2026-09-01T00:00:00Z

Companion guide (§11.4.18) for `scripts/testing/tracker_structural_integrity.sh` and its paired test `scripts/testing/tracker_structural_integrity_test.sh`.

Overview

A §11.4.135 permanent regression guard for one captured defect: on 2026-09-01, during the corpus repair that became commit `8dad4e3`, an ad-hoc removal loop asked to delete four `### G1..G4` item blocks from `Issues.md` ran past the last one and also deleted the **`## H.` section header** and that section's **§11.4.114 preamble**. An independent code review caught it as a BLOCKING finding and it was restored verbatim from `git show HEAD:Issues.md` before the commit landed.

The guard compares each tracker document against its **previous committed revision** and refuses a silent loss of a section or of a section's governing preamble text.

Why the existing machinery did not catch it

`cmd/workable-items` plus `scripts/tests/51_workable_items_db_integrity.sh` already assert that a `sync md-to-db` → `sync db-to-md` round trip is byte-identical. That check cannot see this defect, and the reason is structural rather than incidental: `sync_md_to_db.go` reads each tracker **verbatim** into the `document_sources` table (its own comment: "the verbatim source is what lets db→md replay byte-identical"). Content deleted *before* a sync is therefore absorbed into the DB and replayed back identically — the round trip compares a document to a database derived from that same document, so it stays green on a damaged file. It is a self-referential check, and self-referential checks cannot detect content that vanished before they ran.

`cmd/workable-items/reconcile_test.go` does assert that a preamble survives, but only for the reconciler code path, on a synthetic in-test fixture, for one hard-coded string. Nothing asserted anything about the live corpus.

A search of the gate corpus (`scripts/verify.sh`, `scripts/tests/`, `scripts/challenges/`, `cmd/workable-items/`) found **zero** occurrences of any check that reads a tracker's previous revision (`git show <ref>:Issues.md` or equivalent) and zero shell assertions over `^##` section headers — while a control-needle query for a string known to be present in that same corpus returned 190 matches through the identical search path, so the zero is a real absence and not a blind instrument (§11.4.201(6)).

This defect class has occurred before. `Issues.md` section headers A–E were restored by hand in an earlier cycle ("restored A/B/C/D/E section headers (C and D were missing despite conventions list referencing them)") — recorded in the tracker snapshot under `cmd/workable-items/testdata/`. That makes 2026-09-01 the second documented instance, and both were repaired by a human noticing an absence rather than by any gate.

Prerequisites

- `awk`, `grep`, `sed`, `sort` (POSIX).
- `git` only for the default live-repo mode. Without git the gate emits an explicit `SKIP` with its reason and exits 0 — a detective doc gate that refuses the whole sweep because git is unavailable would be the false-positive refusal §11.4.201(1) forbids.

Usage

```
# Default: Issues.md and Fixed.md, working tree vs HEAD.
bash scripts/testing/tracker_structural_integrity.sh

# Against another revision, or a specific file.
bash scripts/testing/tracker_structural_integrity.sh --baseline
        8dad4e3^
bash scripts/testing/tracker_structural_integrity.sh --file
        Issues.md

# Fixture mode: compare two plain files, no git involved.
bash scripts/testing/tracker_structural_integrity.sh \
  --pair "label:path/to/baseline.md:path/to/current.md"

# The paired test (RED + golden-FALSE + §1.1 mutations).
bash scripts/testing/tracker_structural_integrity_test.sh
```

Exit codes: `0` = PASS or SKIP-with-reason, `1` = at least one assertion FAILed. Every SKIP prints its reason and is never counted as a pass.

What it asserts

ID	Assertion	Fires when
A1	Section survival	A `## <LETTER>.` section present in the baseline is absent from the current file and not declared. Identity is the **letter**, not the title, so retitling a section is legitimate (§11.4.111).
A2	Preamble not emptied	A surviving section whose baseline preamble had content now has none. Binary and threshold-free — rewording and annotating pass, gutting fires.
A3	Anchor survival	A governing `§N.N…` citation present in a baseline preamble is missing from that section's current preamble. This is the clause that catches the exact historical loss: the `## H.` preamble cited §11.4.114.

A section's *preamble* is the content between its `## X.` header and the first `###` item heading (or the next `##` / end of file), with blank lines and `---` rules excluded as scaffolding.

The honest boundary

A2 catches **total** preamble loss, not partial erosion. A partial-erosion rule needs a "how much may disappear" threshold, and no evidence in this repo calibrates one — inventing a number would be the guess §11.4.6 forbids. A3 narrows the gap where it matters most (a governing citation cannot be dropped even by a partial rewrite), and the residual gap is stated here rather than papered over.

The guard also says nothing about whether an item's *content* is correct, whether the DB agrees with the markdown (that is test 51's job), or whether a removal was a good idea (that is the operator's, §11.4.122).

How to remove a section

Removals must be possible; silent removals must not. Two paths:

(a) Tombstone — preferred. Keep the `## X.` header and annotate it as closed. A1 passes naturally, the historical context stays readable, and no manifest row is needed. This is the corpus's own precedent: `## G.` was annotated rather than deleted in `8dad4e3`, and that annotated section is what the guard's `good_repaired.md` fixture reproduces.

(b) Manifest — when the header itself must go. Add a row to `scripts/testing/tracker_section_removals.tsv`:

```
Issues.md<TAB>G<TAB>2026-09-01<TAB>superseded by Fixed.md<TAB>operator, co
```

All five fields must be non-empty. A row with a blank reason or authority is **not** a declaration and the gate still fails — a rubber stamp is not a decision.

Why a checked-in manifest and not an environment flag

The review that caught this defect had to notice an **absence** — that something which should have been in the diff was not. Absence-detection is the hard direction for a reviewer; presence-detection is the easy one. A manifest row lands in the same commit as the removal and turns the removal into an **added line** in the diff, which a reviewer reads directly. An env-var or CLI override would leave no trace in the tree at all, which makes it a bypass rather than a recorded deferral — so the gate deliberately has none.

The tombstone path is listed first because it is strictly better where it applies: it keeps the context a reader of the tracker actually wants, and it needs no second file to stay in sync.

Internal behaviour

Control needle (§11.4.201(6)/(7)(b)). A baseline with zero extracted sections and a genuinely clean comparison both produce "0 missing". Before reporting a clean result the gate prints the number of sections its extractor actually saw, and if the baseline yields zero it reports `BLIND` and SKIPs instead of returning a confident zero.

Two instrument defects found while building this guard, both caught by running the assertions rather than trusting a green summary, and both fixed:

1. `grep -c . "$f" || echo 0` prints `0` **and** exits 1 on an empty file, so the `|| echo 0` appended a second zero. Every later `[ "$n" -gt 0 ]` then died with `integer expected` and evaluated false — **A2 was silently blind**. Replaced with `awk 'NF{n++} END{print n+0}'`, which always emits one integer and always exits 0. The test caught this only because it asserts the *specific* assertion name in the output rather than the exit code alone; an exit-code-only test would have passed on A3's coincidental failure and shipped a blind A2.
2. The T11 mutation's `sed` pattern carried the wrong leading indentation, so the mutation never landed in the copy and the "mutation missed the defect" result would have proven nothing. The test now checks that the mutation

marker is present in the copy *before* trusting the run — a mutation that did not land is a finding, not a pass.

Fixtures. `scripts/testing/fixtures/tracker_structural/` holds the three load-bearing historical shapes as checked-in files (`baseline_issues.md`, `bad_over_deleted.md`, `good_repaired.md`). The four narrower variants (item-added, retitled, item-migrated, preamble-gutted, anchor-lost) are derived inside the test by a single documented transformation each, so a fired assertion can only be attributed to that one transformation — a hand-written variant could differ somewhere else and make the attribution unprovable.

Related scripts

- `scripts/tests/51_workable_items_db_integrity.sh` — the DB round-trip this gate complements. Neither subsumes the other: 51 proves the DB and the markdown agree; this gate proves the markdown did not silently lose content before they were compared.
- `cmd/workable-items/sync_md_to_db.go` — the verbatim absorption that makes the round trip self-referential.
- `scripts/testing/tracker_structural_integrity_test.sh` — the paired test.

Wiring

Not wired into `scripts/verify.sh` by this change: the sweep is owned by another stream and is wired serially by the conductor. Until then the guard runs standalone, and its own test exercises it against the live corpus.

Last verified

2026-09-01 — full test suite `PASS=12 FAIL=0`, three consecutive identical iterations (§11.4.50); RED reproduced on the real `Issues.md` at `8dad4e3^`; golden-FALSE on the real repair commit `8dad4e3^ → 8dad4e3`.